Blockchain Experiment 11

Title:Implementation of Hyperledger Fabric Network and Asset Management using Chaincode

Aim: To implement a Hyperledger Fabric network, deploy chaincode, and perform asset creation, querying, and transfer operations.

Theory:

1. Introduction to Blockchain

Blockchain is a decentralized and distributed digital ledger that records transactions across multiple nodes in a secure and immutable manner. Each block contains a set of transactions linked cryptographically to the previous block, ensuring data integrity and transparency. Unlike public blockchains, enterprise use cases often require permissioned access, privacy, and scalability.

2. Hyperledger Fabric Overview

Hyperledger Fabric is an open-source, enterprise-grade permissioned blockchain framework hosted by the Linux Foundation. It is designed for building secure and scalable blockchain applications with modular architecture.

Key features include:

- Permissioned Network: Only authorized participants can join.
- Modular Design: Components like consensus, membership, and ledger can be customized.
- Privacy & Confidentiality: Supports private channels and data isolation.
- No Cryptocurrency Requirement: Focuses on business applications rather than tokens.

3. Architecture of Hyperledger Fabric

A Hyperledger Fabric network consists of the following components:

- Peers: Nodes that maintain the ledger and execute chaincode.
- Orderers (Ordering Service): Responsible for ordering transactions and creating blocks.
- Channels: Private communication pathways between organizations.
- Membership Service Provider (MSP): Manages identities and authentication.
- Ledger: Contains the world state (current data) and transaction log.

4. Chaincode (Smart Contracts)

In Hyperledger Fabric, smart contracts are called chaincode. Chaincode defines the business logic that governs how assets are created, updated, and transferred within the network.

Key functions of chaincode:

- Defines asset structure (e.g., ID, owner, value)
- Executes transactions (create, read, update, delete)
- Validates rules and permissions
- Ensures consistency across all nodes

Chaincode can be written in languages like Go, JavaScript, or Java.

5. Asset Management in Blockchain

Assets represent anything of value such as goods, property, or digital records. In this experiment, asset management involves:

- Creating Assets: Adding new records to the ledger
- Querying Assets: Retrieving stored data
- Updating Assets: Modifying asset properties
- Deleting Assets: Removing records (logically, as blockchain is immutable)

Each asset transaction is recorded on the ledger, ensuring traceability and transparency.

6. Transaction Flow in Hyperledger Fabric

The typical transaction process includes:

1. Client application sends a transaction proposal.
2. Endorsing peers simulate the transaction using chaincode.
3. Proposal responses are sent back to the client.
4. The ordering service sequences transactions into blocks.
5. Blocks are distributed to peers.
6. Transactions are validated and committed to the ledger.

7. Benefits of Using Hyperledger Fabric for Asset Management

- Enhanced security through cryptographic mechanisms
- Transparency and traceability of transactions
- Decentralized control with no single point of failure
- Fine-grained access control using channels
- Efficient and scalable network performance

8. Applications

- Supply chain management
- Financial services
- Healthcare records management
- Digital identity systems

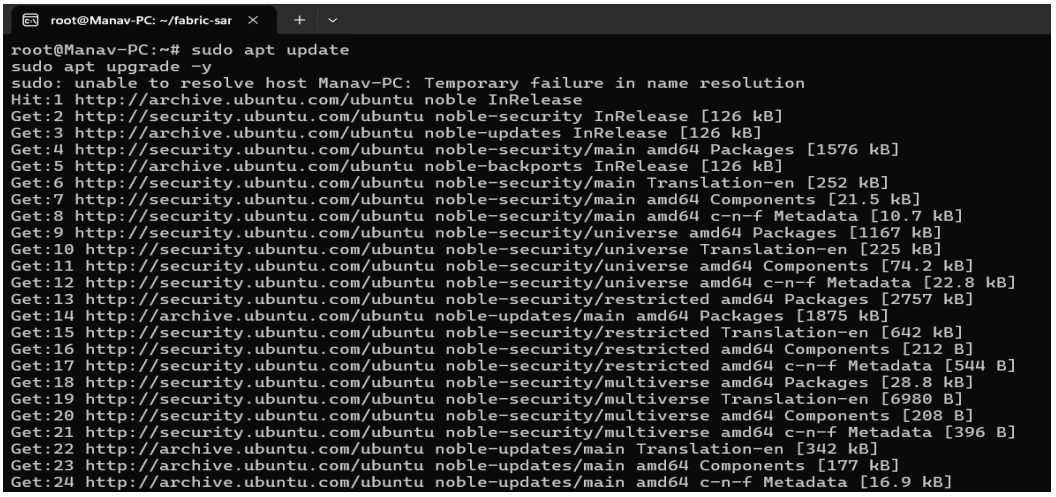
Prerequisites:

- Windows Subsystem for Linux (WSL)
- Ubuntu
- Docker
- Git
- Node.js & npm

Procedure:

Step 1: Update the System

Command:sudo apt update sudo apt upgrade -y



```
root@Manav-PC: ~/fabric-sar
root@Manav-PC:~# sudo apt update
sudo apt upgrade -y
sudo: unable to resolve host Manav-PC: Temporary failure in name resolution
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1576 kB]
Get:5 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:6 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [252 kB]
Get:7 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.5 kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/main amd64 c-n-f Metadata [10.7 kB]
Get:9 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1167 kB]
Get:10 http://security.ubuntu.com/ubuntu noble-security/universe Translation-en [225 kB]
Get:11 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [74.2 kB]
Get:12 http://security.ubuntu.com/ubuntu noble-security/universe amd64 c-n-f Metadata [22.8 kB]
Get:13 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [2757 kB]
Get:14 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1875 kB]
Get:15 http://security.ubuntu.com/ubuntu noble-security/restricted Translation-en [642 kB]
Get:16 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Components [212 B]
Get:17 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 c-n-f Metadata [544 B]
Get:18 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Packages [28.8 kB]
Get:19 http://security.ubuntu.com/ubuntu noble-security/multiverse Translation-en [6980 B]
Get:20 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Components [208 B]
Get:21 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 c-n-f Metadata [396 B]
Get:22 http://archive.ubuntu.com/ubuntu noble-updates/main Translation-en [342 kB]
Get:23 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [177 kB]
Get:24 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 c-n-f Metadata [16.9 kB]
```

Explanation:

This step updates the system package index and upgrades all installed packages to their latest versions. It ensures that the system has the most recent security patches and compatible libraries required for Hyperledger Fabric. Running this step avoids dependency conflicts and installation failures later.

Step 2: Install Required Dependencies

Command:

sudo apt install jq curl git nodejs npm -y

node -v

npm -v

git --version

docker --version

docker ps

```
root@Manav-PC: ~/fabric-sar
root@Manav-PC:~# node -v
v18.19.1
root@Manav-PC:~# npm -v
9.2.0
root@Manav-PC:~# git --version
git version 2.43.0
root@Manav-PC:~# docker --version
Command 'docker' not found, but can be installed with:
snap install docker         # version 28.1.1+1, or
apt  install docker.io      # version 28.2.2-0ubuntu1~24.04.
apt  install podman-docker  # version 4.9.3+ds1-1ubuntu0.2
See 'snap info docker' for additional versions.
root@Manav-PC:~# docker ps
Command 'docker' not found, but can be installed with:
snap install docker         # version 28.1.1+1, or
apt  install docker.io      # version 28.2.2-0ubuntu1~24.04.
apt  install podman-docker  # version 4.9.3+ds1-1ubuntu0.2
See 'snap info docker' for additional versions.
```

Explanation:

This step installs all necessary tools required to run Hyperledger Fabric. Docker is used to run blockchain components as containers, Git is used to clone repositories, and Node.js with npm is required for chaincode execution. Version checks confirm successful installation, while docker ps ensures Docker is running properly.

Step 3: Clone Fabric Samples

Command:

git clone https://github.com/hyperledger/fabric-samples

cd fabric-samples

```
root@Manav-PC:~# git clone https://github.com/hyperledger/fabric-samples
Cloning into 'fabric-samples'...
remote: Enumerating objects: 15618, done.
remote: Counting objects: 100% (216/216), done.
remote: Compressing objects: 100% (124/124), done.
remote: Total 15618 (delta 144), reused 92 (delta 92), pack-reused 15402 (from 3)
Receiving objects: 100% (15618/15618), 24.07 MiB | 4.02 MiB/s, done.
Resolving deltas: 100% (8809/8809), done.
root@Manav-PC:~# cd fabric-samples
root@Manav-PC:~/fabric-samples# curl -sSLO https://raw.githubusercontent.com/hyper
ric.sh
chmod +x install-fabric.sh
```

Explanation:

This step downloads the official Hyperledger Fabric sample repository. It contains pre-built network configurations, scripts, and example chaincodes. Moving into this directory allows us to access all required files for setting up the blockchain network.

Step 4: Install Fabric Binaries and Images

Command:

curl -sSLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh

chmod +x install-fabric.sh

./install-fabric.sh docker samples binary

./install-fabric.sh docker

```
root@Manav-PC:~/fabric-samples# curl -sSLO https://raw.githubusercontent.com/hyperledge
ric.sh
chmod +x install-fabric.sh
./install-fabric.sh docker samples binary
./install-fabric.sh docker

Clone hyperledger/fabric-samples repo

==> Already in fabric-samples repo
fabric-samples v2.5.15 does not exist, defaulting to main. fabric-samples main branch i
rsions of fabric.

Pull Hyperledger Fabric binaries

===> Downloading version 2.5.15 platform specific fabric binaries
===> Downloading:  https://github.com/hyperledger/fabric/releases/download/v2.5.15/hype
.tar.gz
===> Will unpack to: /root/fabric-samples
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
100  120M  100  120M    0     0  3701k      0  0:00:33  0:00:33 --:--:-- 3516k
==> Done.
```

```
Pull Hyperledger Fabric docker images

FABRIC_IMAGES: peer orderer ccenv baseos
===> Pulling fabric Images
====>  ghcr.io/hyperledger/fabric-peer:2.5.15
2.5.15: Pulling from hyperledger/fabric-peer
Digest: sha256:3e25adc31a757ee19dd8a24afdc9ac5aa46a86b3cb56fc93068b0e594706d6aa
Status: Image is up to date for ghcr.io/hyperledger/fabric-peer:2.5.15
ghcr.io/hyperledger/fabric-peer:2.5.15
====>  ghcr.io/hyperledger/fabric-orderer:2.5.15
2.5.15: Pulling from hyperledger/fabric-orderer
Digest: sha256:9972c209be47f45e377a24c88f2e3d21fae4dc224751c68bdd33f2e7a603ffa8
Status: Image is up to date for ghcr.io/hyperledger/fabric-orderer:2.5.15
ghcr.io/hyperledger/fabric-orderer:2.5.15
====>  ghcr.io/hyperledger/fabric-ccenv:2.5.15
2.5.15: Pulling from hyperledger/fabric-ccenv
Digest: sha256:86dded7eda9268e2cbf3691cb952edf545dc5faea1a79bfc9b47ec47d3ecfa9e
Status: Image is up to date for ghcr.io/hyperledger/fabric-ccenv:2.5.15
ghcr.io/hyperledger/fabric-ccenv:2.5.15
====>  ghcr.io/hyperledger/fabric-baseos:2.5.15
2.5.15: Pulling from hyperledger/fabric-baseos
Digest: sha256:654bd4554bf97c70902e56d530294795372518da7ae2a7cdfccbe397a878c513
Status: Image is up to date for ghcr.io/hyperledger/fabric-baseos:2.5.15
ghcr.io/hyperledger/fabric-baseos:2.5.15
===> Pulling fabric ca Image
====>  ghcr.io/hyperledger/fabric-ca:1.5.17
```

Explanation:

This step downloads and executes a script that installs Hyperledger Fabric binaries and required Docker images. It pulls essential components like peer, orderer, and certificate authority images. These components are necessary to create and manage the blockchain network.

Step 5: Verify Docker Images

Command: docker images | grep fabric

```
root@Manav-PC:~/fabric-samples# docker images | grep fabric
WARNING: This output is designed for human readability. For machine-readable output,
ghcr.io/hyperledger/fabric-baseos:2.5.15    654bd4554bf9        250MB         80.4MB
ghcr.io/hyperledger/fabric-ca:1.5.17        453a0a727e82        381MB          123MB
ghcr.io/hyperledger/fabric-ccenv:2.5.15     86dded7eda92          1GB          255MB
ghcr.io/hyperledger/fabric-orderer:2.5.15   9972c209be47        178MB         49.5MB
ghcr.io/hyperledger/fabric-peer:2.5.15      3e25adc31a75        230MB         66.2MB
hyperledger/fabric-baseos:2.5               654bd4554bf9        250MB         80.4MB
hyperledger/fabric-baseos:2.5.15            654bd4554bf9        250MB         80.4MB
hyperledger/fabric-baseos:latest            654bd4554bf9        250MB         80.4MB
hyperledger/fabric-ca:1.5                   453a0a727e82        381MB          123MB
hyperledger/fabric-ca:1.5.17                453a0a727e82        381MB          123MB
hyperledger/fabric-ca:latest                453a0a727e82        381MB          123MB
hyperledger/fabric-ccenv:2.5                86dded7eda92          1GB          255MB
hyperledger/fabric-ccenv:2.5.15             86dded7eda92          1GB          255MB
hyperledger/fabric-ccenv:latest             86dded7eda92          1GB          255MB
hyperledger/fabric-orderer:2.5              9972c209be47        178MB         49.5MB
hyperledger/fabric-orderer:2.5.15           9972c209be47        178MB         49.5MB
hyperledger/fabric-orderer:latest           9972c209be47        178MB         49.5MB
hyperledger/fabric-peer:2.5                 3e25adc31a75        230MB         66.2MB
hyperledger/fabric-peer:2.5.15              3e25adc31a75        230MB         66.2MB
hyperledger/fabric-peer:latest              3e25adc31a75        230MB         66.2MB
```

Explanation:

This command checks whether all required Hyperledger Fabric Docker images are successfully downloaded.

It ensures that components such as peer and orderer are available before starting the network. Missing images may cause deployment errors.

Step 6: Set Environment Variables

Command:

export PATH=$PATH:$HOME/fabric-samples/bin

export FABRIC_CFG_PATH=$HOME/fabric-samples/config

peer version

```
root@Manav-PC:~/fabric-samples# export PATH=$PATH:$HOME/fabric-samples/bin
root@Manav-PC:~/fabric-samples# export FABRIC_CFG_PATH=$HOME/fabric-samples/config
root@Manav-PC:~/fabric-samples# peer version
peer:
 Version: v2.5.15
 Commit SHA: 83c7930
 Go version: go1.26.0
 OS/Arch: linux/amd64
 Chaincode:
  Base Docker Label: org.hyperledger.fabric
  Docker Namespace: hyperledger
```

Explanation:

This step sets environment variables required to run Fabric CLI commands. The PATH variable allows access to Fabric binaries, while FABRIC_CFG_PATH points to configuration files. Running peer version verifies that the setup is successful.

Step 7: Start Network and Create Channel

Command:

cd ~/fabric-samples/test-network

./network.sh up createChannel

```
root@Manav-PC:~/fabric-samples# cd ~/fabric-samples/test-network
./network.sh up createChannel
Using docker and docker-compose
Creating channel 'mychannel'.
If network is not up, starting nodes with CLI timeout of '5' tries and CLI delay of '3' seconds and using d
ldb with crypto from 'cryptogen'
Bringing up network
LOCAL_VERSION=v2.5.15
DOCKER_IMAGE_VERSION=v2.5.15
/root/fabric-samples/test-network/../bin/cryptogen
Generating certificates using cryptogen tool
Creating Org1 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org1.yaml --output=organizations
org1.example.com
+ res=0
Creating Org2 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org2.yaml --output=organizations
org2.example.com
+ res=0
Creating Orderer Org Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-orderer.yaml --output=organizations
+ res=0
Generating CCP files for Org1 and Org2
[+] up 7/7
 ✔Network fabric_test                    Created
 ✔Volume compose_orderer.example.com     Created
```

```
root@Manav-PC: ~/fabric-sar

+ peer lifecycle chaincode commit -o localhost:7050 --ordererTLSHostnameOverride orde
t/fabric-samples/test-network/organizations/ordererOrganizations/example.com/tlsca/tl
D mychannel --name basic --peerAddresses localhost:7051 --tlsRootCertFiles /root/fabr
ns/peerOrganizations/org1.example.com/tlsca/tlsca.org1.example.com-cert.pem --peerAdd
Files /root/fabric-samples/test-network/organizations/peerOrganizations/org2.example.
rt.pem --version 1.0 --sequence 1
+ res=0
2026-04-03 13:22:50.210 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [136f63476f31
689af6a0c344c441b] committed with status (VALID) at localhost:9051
2026-04-03 13:22:50.215 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid [136f63476f31
689af6a0c344c441b] committed with status (VALID) at localhost:7051
Chaincode definition committed on channel 'mychannel'
Using organization 1
Querying chaincode definition on peer0.org1 on channel 'mychannel'...
Attempting to Query committed status on peer0.org1, Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approva
Query chaincode definition successful on peer0.org1 on channel 'mychannel'
Using organization 2
Querying chaincode definition on peer0.org2 on channel 'mychannel'...
Attempting to Query committed status on peer0.org2, Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
```

Explanation:
This step initializes the blockchain network by starting Docker containers for peers and orderers. It also creates a communication channel where transactions will occur. Certificates and configurations are automatically generated during this process.

Step 8: Deploy Chaincode
Command:
./network.sh deployCC \
  -ccn basic \
  -ccp ../asset-transfer-basic/chaincode-javascript \
  -ccl javascript

```
root@Manav-PC:~/fabric-samples# cd ~/fabric-samples/test-network
root@Manav-PC:~/fabric-samples/test-network# ./network.sh deployCC \
  -ccn basic \
  -ccp ../asset-transfer-basic/chaincode-javascript \
  -ccl javascript
Using docker and docker-compose
deploying chaincode on channel 'mychannel'
executing with the following
- CHANNEL_NAME: mychannel
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
- CC_END_POLICY: NA
- CC_COLL_CONFIG: NA
- CC_INIT_FCN: NA
- DELAY: 3
- MAX_RETRY: 5
- VERBOSE: false
executing with the following
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
+ '[' false = true ']'
+ peer lifecycle chaincode package basic.tar.gz --path ../asset-transfer-bas
basic_1.0
+ res=0
Chaincode is packaged
```

Explanation:
This step deploys the smart contract (chaincode) onto the blockchain network. The chaincode defines the business logic for asset operations such as creation, transfer, and querying. Once deployed, it is ready to interact with the ledger.

Step 9: Set Peer Environment
export FABRIC_CFG_PATH=$HOME/fabric-samples/config
export PATH=$PATH:$HOME/fabric-samples/bin
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export
CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export
CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051

```
root@Manav-PC:~/fabric-samples/test-network# export FABRIC_CFG_PATH=$HOME/fabric-samples/config
export PATH=$PATH:$HOME/fabric-samples/bin
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/
ers/peer0.org1.example.com/tls/ca.crt
export CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1
Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051
```

Explanation:
This step configures the identity and communication settings of the peer node. It specifies organization details, TLS certificates, and network addresses. These settings are required for secure interaction with the blockchain network.

Step 10: Initialize Ledger
Command:
peer chaincode invoke ... InitLedger

```
root@Manav-PC:~/fabric-samples/test-network# peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/ordere
/msp/tlscacerts/tlsca.example.com-cert.pem \
  -C mychannel \
  -n basic \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/pee
.example.com/tls/ca.crt \
  --peerAddresses localhost:9051 \
  --tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/pee
.example.com/tls/ca.crt \
  --waitForEvent \
  -c '{"function":"InitLedger","Args":[]}'
2026-04-03 13:27:01.786 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [739a6ae87df069979d1bc20a6341876226
c12b3bd8f2591250b] committed with status (VALID) at localhost:7051
2026-04-03 13:27:01.787 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid [739a6ae87df069979d1bc20a6341876226
c12b3bd8f2591250b] committed with status (VALID) at localhost:9051
2026-04-03 13:27:01.790 UTC 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful.
```

Explanation:
This step initializes the blockchain ledger with default sample assets. It writes initial data to the ledger, which can later be queried or modified. This operation is a transaction that gets recorded on the blockchain.

Step 11: Query All Assets
Command:
peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'

```
root@Manav-PC:~/fabric-samples/test-network# peer chaincode query \
  -C mychannel \
  -n basic \
  -c '{"Args":["GetAllAssets"]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"docType":"asset"},{"AppraisedValue":400,"
Color":"red","ID":"asset2","Owner":"Brad","Size":5,"docType":"asset"},{"AppraisedValue":500,"Color":"green","ID":"asset3
","Owner":"Jin Soo","Size":10,"docType":"asset"},{"AppraisedValue":600,"Color":"yellow","ID":"asset4","Owner":"Max","Siz
e":10,"docType":"asset"},{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asse
t"},{"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Michel","Size":15,"docType":"asset"}]
```

Explanation:
This step retrieves all assets stored in the ledger. It performs a read-only operation and displays current data without modifying the blockchain. It helps verify that the ledger has been initialized correctly.

Step 12: Read Specific Asset
Command:
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset5"]}'

```
root@Manav-PC:~/fabric-samples/test-network# peer chaincode query \
  -C mychannel \
  -n basic \
  -c '{"Args":["ReadAsset","asset5"]}'
{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"}
```

Explanation:
This step retrieves detailed information of a specific asset using its ID. It demonstrates how individual records can be accessed from the ledger. This is also a read-only operation.

Step 13: Create New Asset
Command:
peer chaincode invoke ... CreateAsset (Owner: Manav)

```
root@Manav-PC:~/fabric-samples/test-network# peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com
/msp/tlscacerts/tlsca.example.com-cert.pem \
  -C mychannel \
  -n basic \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1
.example.com/tls/ca.crt \
  --peerAddresses localhost:9051 \
  --tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2
.example.com/tls/ca.crt \
  --waitForEvent \
  -c '{"function":"CreateAsset","Args":["asset10","blue","10","Manav","1000"]}'
2026-04-03 13:37:44.489 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [c0fb2f13a0c62f343bed8b6ae7e7fc02db99ba8f961f539
2ad0df3b16321450e] committed with status (VALID) at localhost:7051
2026-04-03 13:37:44.490 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid [c0fb2f13a0c62f343bed8b6ae7e7fc02db99ba8f961f539
2ad0df3b16321450e] committed with status (VALID) at localhost:9051
2026-04-03 13:37:44.490 UTC 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: stat
us:200 payload:"{\"ID\":\"asset10\",\"Color\":\"blue\",\"Size\":10,\"Owner\":\"Manav\",\"AppraisedValue\":1000}"
```

Explanation:

This step creates a new asset on the blockchain with specified attributes such as ID, color, size, owner, and value. The owner name is set to Manav, which confirms that the transaction was performed by the user. This operation updates the ledger.

Step 14: Transfer Asset

Command:

peer chaincode invoke ... TransferAsset

```
root@Manav-PC:~/fabric-samples/test-network# peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com
/msp/tlscacerts/tlsca.example.com-cert.pem \
  -C mychannel \
  -n basic \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1
.example.com/tls/ca.crt \
  --peerAddresses localhost:9051 \
  --tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2
.example.com/tls/ca.crt \
  --waitForEvent \
  -c '{"function":"TransferAsset","Args":["asset10","Rahul"]}'
2026-04-03 13:41:46.421 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [223f359d1497e07db6d93c10e9a6c2917665628376992f7
bacafe5ad0d3aaadd] committed with status (VALID) at localhost:9051
2026-04-03 13:41:46.424 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid [223f359d1497e07db6d93c10e9a6c2917665628376992f7
bacafe5ad0d3aaadd] committed with status (VALID) at localhost:7051
2026-04-03 13:41:46.426 UTC 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: stat
us:200 payload:"Manav"
```

Explanation:

This step transfers ownership of an asset from one user to another. It updates the owner field in the ledger. This demonstrates how blockchain ensures secure and traceable asset transfers.

Step 15: Stop Network

Command:

./network.sh down

```
root@Manav-PC:~/fabric-samples/test-network# ./network.sh down
Using docker and docker-compose
Stopping network
[+] down 7/7
 ✔Container peer0.org1.example.com      Removed
 ✔Container peer0.org2.example.com      Removed
 ✔Container orderer.example.com         Removed
 ✔Volume compose_orderer.example.com    Removed
 ✔Network fabric_test                   Removed
 ✔Volume compose_peer0.org1.example.com Removed
 ✔Volume compose_peer0.org2.example.com Removed
Error response from daemon: get docker_orderer.example.com: no such volume
Error response from daemon: get docker_peer0.org1.example.com: no such volume
Error response from daemon: get docker_peer0.org2.example.com: no such volume
Removing remaining containers
6152d81175c6
f5b73b9b7204
Removing generated chaincode docker images
Untagged: dev-peer0.org2.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbe664c87b24c035bee15cbcc7f5
2a0ff0cc1497615f5e33a8bfa908e0fd3555f864cfa1398b52d341cb:latest
Deleted: sha256:54e17cd9d88b065838b9b4184da44258c6c5402e7b2aa9ee3a324fc09cdd84fd
Untagged: dev-peer0.org1.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbe664c87b24c035bee15cbcc7f5
4b9747cc8433ac0e74bfaeade1cf0b97f5c1163835f604aa885d60dc:latest
Deleted: sha256:4b8656488ea2c9620f5669d82c6bee43ad10e6a71bb5f57cde69bfdb57fcd860
```

Explanation:
This step stops all running Docker containers and shuts down the Hyperledger Fabric network. It frees system resources and safely ends the experiment.

Result:
The Hyperledger Fabric network was successfully implemented. Chaincode was deployed and asset operations such as creation, querying, and transfer were performed successfully. The asset was created with owner name Manav, confirming correct execution.

Conclusion:

This experiment demonstrated how blockchain networks operate using Hyperledger Fabric and provided practical insight into how transactions are securely executed within a permissioned environment. Through the setup and configuration of the network, we gained a clear understanding of key components such as peers, ordering services, and channels, and how they interact to maintain a distributed ledger.

The experiment also helped in understanding real-time asset management using smart contracts (chaincode), where assets can be created, updated, queried, and transferred efficiently while ensuring data integrity and transparency. It highlighted how business logic is embedded within the blockchain to automate processes and enforce rules without the need for intermediaries.Additionally, we observed how transactions are validated, endorsed, and committed to the ledger, ensuring consistency across all participating nodes. The hands-on implementation reinforced concepts such as decentralization, security, immutability, and traceability of data. Overall, the experiment provided a comprehensive understanding of how Hyperledger Fabric can be used to build secure, scalable, and efficient enterprise-grade blockchain applications for asset management.